

5.[Pracess management Based practical]

In an operating system three CPU-intensive processes are ready for execution, which require 10ns, 20ns and 30ns and arrival at times 0ns, 2ns and 6ns, respectively. Write a Program to calculate the total number of context switches needed if the operating system implements a shortest job first (preemptive) scheduling algorithm. Also calculate the average time for which the processes have to wait before getting the CPU.

CODE:

```
GNU nano 8.7                                New Buffer                                Modified
#include <stdio.h>

int main() {
    int n = 3;
    int at[3] = {0, 2, 6}; // Arrival times
    int bt[3] = {10, 20, 30}; // Burst times
    int rt[3]; // Remaining times
    int wt[3] = {0};
    int ct[3];

    int time = 0, completed = 0;
    int prev = -1;
    int context_switch = 0;

    for (int i = 0; i < n; i++)
        rt[i] = bt[i];

    while (completed < n) {
        int shortest = -1;
        int min_rt = 9999;

        for (int i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0 && rt[i] < min_rt) {
                min_rt = rt[i];
                shortest = i;
            }
        }

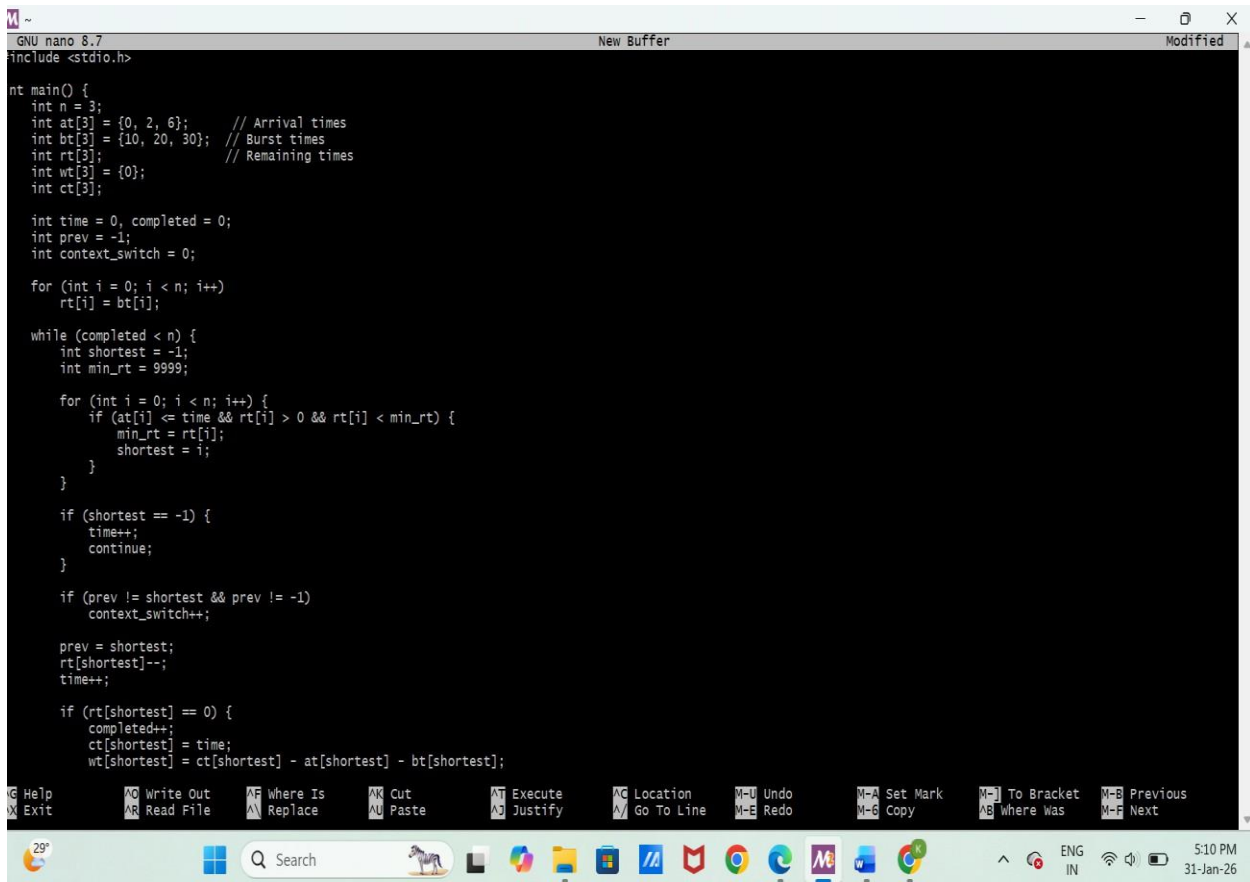
        if (shortest == -1) {
            time++;
            continue;
        }

        if (prev != shortest && prev != -1)
            context_switch++;

        prev = shortest;
        rt[shortest]--;
        time++;

        if (rt[shortest] == 0) {
            completed++;
            ct[shortest] = time;
            wt[shortest] = ct[shortest] - at[shortest] - bt[shortest];
        }
    }

    printf("Total context switches: %d\n", context_switch);
    printf("Average waiting time: %f\n", (wt[0] + wt[1] + wt[2]) / n);
}
```



```
GNU nano 8.7 New Buffer Modified
while (completed < n) {
    int shortest = -1;
    int min_rt = 9999;

    for (int i = 0; i < n; i++) {
        if (at[i] <= time && rt[i] > 0 && rt[i] < min_rt) {
            min_rt = rt[i];
            shortest = i;
        }
    }

    if (shortest == -1) {
        time++;
        continue;
    }

    if (prev != shortest && prev != -1)
        context_switch++;

    prev = shortest;
    rt[shortest]--;
    time++;

    if (rt[shortest] == 0) {
        completed++;
        ct[shortest] = time;
        wt[shortest] = ct[shortest] - at[shortest] - bt[shortest];
    }

    float avg_wt = 0;
    printf("\nProcess\tAT\tBT\tWT\n");
    for (int i = 0; i < n; i++) {
        avg_wt += wt[i];
        printf("P%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i]);
    }

    printf("\nTotal Context Switches = %d", context_switch);
    printf("\nAverage Waiting Time = %.2f ns\n", avg_wt / n);

    return 0;
}
```

Output:

```
Ansh@Anshs-Rog MSYS ~
$ nano sjf_primitive.c

Ansh@Anshs-Rog MSYS ~
$ gcc sjf_primitive.c -o sjf

Ansh@Anshs-Rog MSYS ~
$ ./sjf

Process  AT          BT          WT
P1       0           10           0
P2       2           20           8
P3       6           30          24

Total Context Switches = 2
Average Waiting Time = 10.67 ns

Ansh@Anshs-Rog MSYS ~
$ |
```